Dot Net Core and Progressive Web Application

Module 1: Introduction to C#.NET | Module 2: PWA

Academic Year 2026 – 2027

📚 Introduction to the Subject

What is this Subject About?

This subject is divided into two interrelated modules that together cover full-stack modern web application development using Microsoft's .NET ecosystem on the backend and Progressive Web Application (PWA) technology on the frontend.

By the end of this course, students will be able to build, deploy, and optimize a complete web application — from server-side business logic written in C# to a fast, installable, offline-capable web frontend.

Module Overview

Module	Name	Focus Area
Module 1	Introduction to C#.NET	Language basics, OOP, ASP.NET Core, REST APIs
Module 2	Progressive Web Applications (PWA)	Service Workers, Manifest, Caching, .NET Integration

Think of it as a journey:

1. **First**, you learn the C# programming language.
2. **Then**, you learn Object-Oriented Programming (OOP) concepts using C#.
3. **After that**, you learn ASP.NET Core, which is used to build websites, web applications, and APIs.
4. **Finally**, you learn PWA technology, which allows websites to behave like mobile applications.

Why Are We Studying This?

The combination of C#.NET and PWA represents one of the most powerful and in-demand stacks in enterprise software development today. Here is why this matters:

- Microsoft .NET powers over 35% of enterprise backend systems globally.
- C# consistently ranks in the Top 5 most-used programming languages (TIOBE Index).
- PWAs are now supported by all major browsers and are used by companies like Twitter, Starbucks, Pinterest, and Uber.
- Learning this stack opens career paths in full-stack development, cloud computing (Azure), and enterprise application development.

Real-World Usage & Industry Applications

Technology	Companies Using It	Use Cases
C# / .NET	Microsoft, Stack Overflow, GoDaddy, Accenture	Web APIs, enterprise apps, cloud microservices
ASP.NET Core	Samsung, TuneIn, GE Aviation	High-performance REST APIs, real-time apps
PWA	Twitter Lite, Starbucks, Pinterest, Uber	Offline web apps, push notifications, installable web

1. C# = The Language

Just like English is used to write sentences, **C#** is used to write computer programs.
Example:
Console.WriteLine("Hello World");
Here, C# is the language used to give instructions to the computer.

2. .NET = The Platform

If C# is the language, then **.NET** is the environment that helps run C# programs.
Without .NET, the computer would not understand C# code directly.
Analogy:

- English → Language
- Human Brain → Understands English

Similarly,

- C# → Programming Language
- .NET → Understands and runs C# code

3. ASP.NET Core = Web Development Framework

Suppose you want to create a website like:

- College Portal
- Hospital Management System
- Online Shopping Website

You use **ASP.NET Core**.
It is a framework inside .NET specifically designed for web applications.
Hierarchy:
C# → Language
.NET → Platform
ASP.NET Core → Web Framework inside .NET
Example:

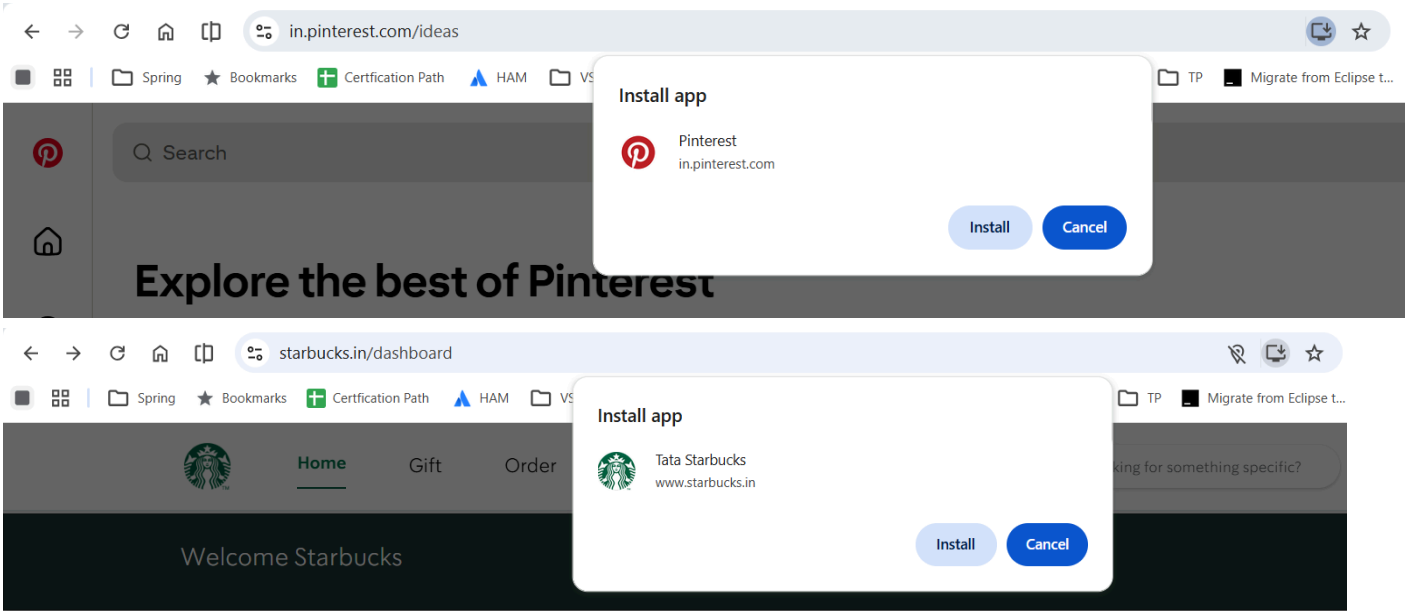
Instagram Website
↓
ASP.NET Core
↓
.NET
↓
C#

4. PWA (Progressive Web App) = Type of Application

PWA is **not a language** and **not a framework**.
It is a special type of web application that behaves like a mobile app.
Examples:

- Installable from browser
- Works offline
- Sends notifications

Example:
Normal Website:
www.example.com
PWA:
www.example.com
+
Can be installed on phone
+
Works offline
+
Looks like an app



MODULE 1

Introduction to C#.NET

Chapter 1: Overview of the .NET Ecosystem

Before writing a single line of C# code, it is essential to understand the environment in which C# runs — the .NET ecosystem. Think of .NET as the entire infrastructure that supports your program: runtime, libraries, tools, and deployment pipelines.

1.1 What is .NET?

.NET (pronounced "dot net") is a free, open-source, cross-platform developer platform created by Microsoft. It provides:

- A runtime environment (CLR — Common Language Runtime) that executes programs
- A huge standard library of pre-built code (Base Class Library / BCL)
- Compilers and tools to build, test, and deploy applications
- Support for multiple programming languages: C#, F#, Visual Basic

> 📌 **Note:** Full Form: .NET does NOT stand for anything — it is a brand name. However, CLR = Common Language Runtime, BCL = Base Class Library, SDK = Software Development Kit, CLI = Command Line Interface.

1.2 Evolution of .NET: Framework vs. Core vs. 5+

Understanding the history of .NET is crucial for working in real-world projects where legacy code exists alongside modern codebases.

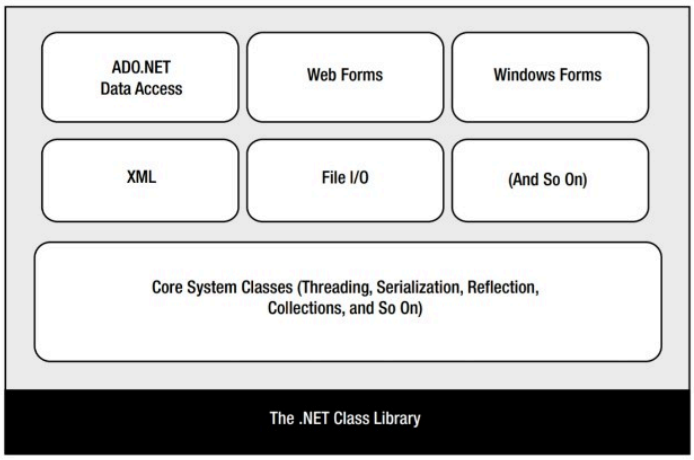
Feature	.NET Framework	.NET Core	.NET 5 / 6	.NET 7 / 8+
Released	2002	2016	2020 / 2021	2022 / 2023
Platform	Windows Only	Cross-platform	Cross-platform	Cross-platform
Open Source	Partially	Yes	Yes	Yes
Status	Legacy (4.8 = last)	Merged into .NET 5	Supported	Current / LTS
Performance	Moderate	High	Very High	Highest

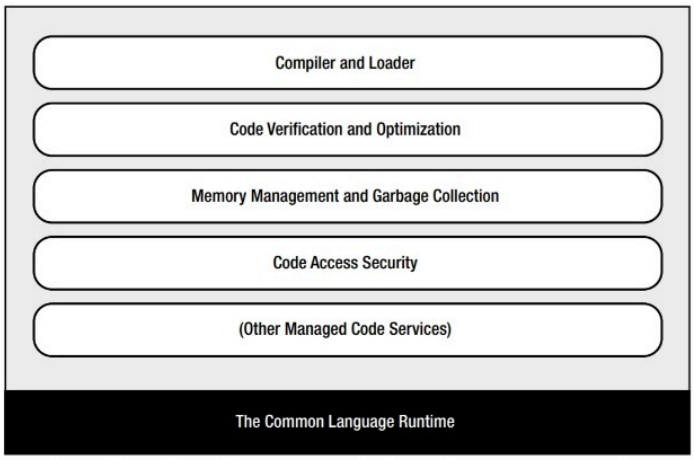
> **Key Insight:** .NET 5 unified .NET Framework and .NET Core into a single platform. From .NET 5 onwards, the word 'Core' was dropped. Today, .NET 8 is the Long-Term Support (LTS) version and is the recommended version for all new projects.

Visual Timeline of .NET Evolution

```
.NET EVOLUTION TIMELINE
═══════════════════════════════════════════════════════════════
2002 ──── .NET Framework 1.0   (Windows Only)          [LEGACY]
2016 ──── .NET Core 1.0         (Cross-platform)        [MODERN]
2019 ──── .NET Core 3.1         (LTS, last Core)        [STABLE]
2020 ──── .NET 5                (Unified Platform)      [UNIFIED]
2021 ──── .NET 6                (LTS - 3 year support)  [CURRENT]
2022 ──── .NET 7                (STS - 18 months)       [STABLE]
2023 ──── .NET 8                (LTS - Recommended)     [BEST]
2024 ──── .NET 9                (STS)                   [LATEST]
═══════════════════════════════════════════════════════════════
LTS = Long Term Support (3 years)  |  STS = Standard Term (18 months)
```

.NET Framework architecture





1. .NET Class Library

The upper part of the diagram shows the **.NET Class Library**, which is a collection of pre-written classes and functions that developers can use.

Components:

ADO.NET ActiveX Data Objects (Data Access)

- Used to connect to databases.
- Helps perform operations like:
    - Insert

    - Update
    - Delete
    - Retrieve data

Example: Connecting a C# application to SQL Server.

Web Forms

- Used for developing web applications.
- Allows creation of web pages using ASP.NET.

Example: Online registration forms, login pages.

Windows Forms

- Used to create desktop applications with graphical user interfaces (GUI).

Example: Calculator, Notepad-like applications.

XML

- Used to store and exchange data in a structured format.

Example:

```
<Student>
  <Name>Swati</Name>
</Student>
```

File I/O

- Used to read and write files.

Example:

- Reading a text file
- Saving user data into a file

Core System Classes

These are the basic classes used by all .NET applications.
They provide:

- Threading (Multitasking)
- Serialization (Object storage)
- Reflection (Inspecting code at runtime)
- Collections (List, ArrayList, Dictionary)

Example:

```
List<string> names = new List<string>();
```

2. Common Language Runtime (CLR)

The lower part of the figure shows the **CLR**, which is the execution engine of .NET.
Think of CLR as the **heart of the .NET Framework**.
When a C# program runs, CLR manages everything.

Compiler and Loader

- Converts Intermediate Language (IL) code into machine code.
- Loads assemblies into memory.

Function: Makes the program executable.

Code Verification and Optimization

- Checks whether code is safe.
- Optimizes code for better performance.

Benefit: Faster execution and fewer errors.

Memory Management and Garbage Collection

- Automatically allocates and deallocates memory.
- Removes unused objects.

Benefit: Programmer does not need to free memory manually.
Example:
Student s = new Student();
When s is no longer needed, Garbage Collector removes it automatically.

Code Access Security

- Controls what resources a program can access.
- Protects the system from unsafe code.

Example:

- Restrict file access
- Restrict network access

Other Managed Code Services
CLR also provides:

- Exception handling
- Type checking
- Debugging support
- Security services

Working Flow of .NET Framework
C# Program
↓
Compiler
↓
Intermediate Language (IL)
↓
CLR
↓
Machine Code
↓
Execution

1.3 Common Language Runtime (CLR)

The CLR is the heart of the .NET platform. It is the virtual machine (VM) that runs your C# program. When you write C# code and compile it, it does NOT compile directly to machine code (0s and 1s). Instead, it goes through a two-step process:

Step 1: C# Source → CIL (Common Intermediate Language)

The C# compiler (csc.exe or Roslyn) translates your code into CIL (also called MSIL — Microsoft Intermediate Language). CIL is a CPU-independent instruction set — it is not tied to any specific processor.

Step 2: CIL → Native Machine Code via JIT Compiler

When you run the program, the CLR's JIT (Just-In-Time) compiler translates CIL into native machine code specific to your CPU architecture (x64, ARM, etc.).

What Else Does the CLR Do?

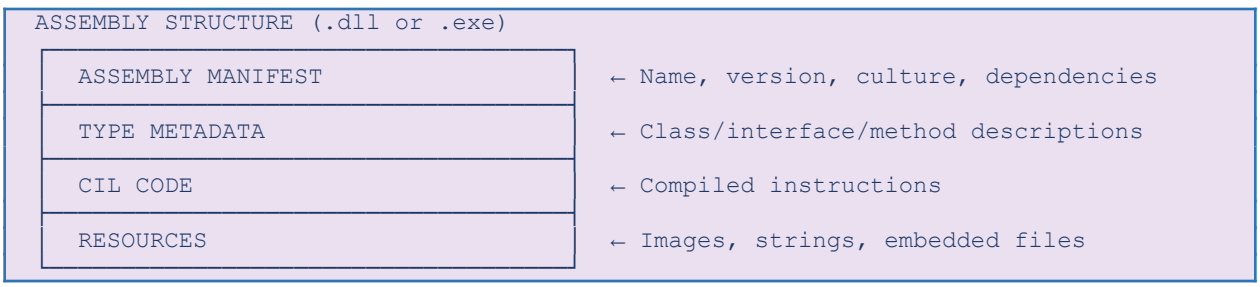
CLR Feature	What It Does
Garbage Collector (GC)	Automatically frees memory that is no longer used — no manual memory management like in C/C++
Exception Handling	Provides structured error handling using try/catch/finally blocks
Security Manager	Controls what code is allowed to do (file access, network, etc.)
Type Safety	Ensures variables are used consistently with their declared types
Thread Management	Manages multi-threading and parallel execution

1.4 Assemblies

An Assembly is the compiled output of your .NET project. It is either a .dll (library) or .exe (executable) file that contains CIL code, metadata (type information), and resources.

Types of Assemblies

- Private Assembly — used only by a single application (placed in app folder)
- Shared Assembly — stored in the GAC (Global Assembly Cache) and shared across multiple applications
- Satellite Assembly — contains only resources like language-specific strings (used in localization)



1.5 .NET CLI (Command Line Interface) Tools

The .NET CLI is a cross-platform tool that lets you create, build, run, and publish .NET applications directly from the terminal without needing Visual Studio.

What are CLI Tools?

CLI (Command Line Interface) Tools are tools that allow developers to create, build, run, test, debug, and publish .NET applications using commands instead of a graphical interface like Visual Studio.

Instead of clicking buttons, you type commands in the Command Prompt, PowerShell, Terminal, or VS Code Terminal.

```
C:\Users\msmar>dotnet

Usage: dotnet [path-to-application]
Usage: dotnet [commands]

path-to-application:
  The path to an application .dll file to execute.

commands:
  -h|--help                         Display help.
  --info                            Display .NET information.
  --list-runtimes [--arch <arch>]   Display the installed runtimes matching the host or specified architecture. Example
architectures: arm64, x64, x86.
  --list-sdks [--arch <arch>]       Display the installed SDKs matching the host or specified architecture. Example arch
itectures: arm64, x64, x86.
```

Common .NET CLI Commands

CLI Command	What It Does
dotnet --version	Show installed SDK version
dotnet --info	Display SDK and runtime information
dotnet new console	Create a console application
dotnet new mvc	Create an ASP.NET MVC application
dotnet new webapi	Create an ASP.NET Core Web API
dotnet new classlib	Create a class library
dotnet new list	List available project templates
dotnet build	Compile the project
dotnet run	Build (if needed) and run the project
dotnet clean	Remove build artifacts (bin and obj)
dotnet restore	Restore NuGet package dependencies
dotnet add package <PackageName>	Install a NuGet package
dotnet remove package <PackageName>	Remove a NuGet package
dotnet list package	Display installed NuGet packages

dotnet test	Run unit tests
dotnet publish	Create files for deployment
dotnet help	Show general help
dotnet <command> --help	Show help for a specific command

> 📌 **Note:** NuGet is the package manager for .NET — similar to npm for JavaScript or pip for Python. It hosts thousands of free, open-source libraries you can add to your project.

Chapter 2: Basics of C# Language

https://visualstudio.microsoft.com/vs/community/

C# (pronounced 'C Sharp') is a modern, object-oriented, type-safe programming language developed by Microsoft. It was designed by Anders Hejlsberg (who also created TypeScript and Turbo Pascal). C# combines the power of C++ with the simplicity of Java.

2.1 Data Types in C#

Every variable in C# has a data type — it tells the compiler what kind of data the variable will hold and how much memory to allocate.

Value Types (stored directly in memory stack)

Type	Size	Range / Example	Usage
int	4 bytes	-2B to +2B	Age, count, scores: int age = 21;
long	8 bytes	Very large numbers	File size, timestamps: long bytes = 1234567890L;
double	8 bytes	15-17 decimal digits	Prices, weights: double price = 99.99;
float	4 bytes	6-9 decimal digits	Graphics coordinates: float x = 3.14f;
decimal	16 bytes	28-29 digits precision	Financial: decimal salary = 50000.00m;
bool	1 byte	true / false	Flags: bool isActive = true;
char	2 bytes	Unicode character	Single letter: char grade = 'A';
byte	1 byte	0 to 255	Pixel values, binary data

Reference Types (stored in heap, variable holds a reference/pointer)

Type	Description & Example
string	Sequence of characters: string name = "Rahul Sharma";
object	Base type for ALL types in C#: object obj = 42;
array	Fixed-size collection: int[] marks = {85, 90, 78};
class	User-defined reference type: Student s = new Student();

> 📌 **Note:** string in C# is an alias for System.String. Similarly, int is an alias for System.Int32. C# provides short aliases for all common types for convenience.

2.2 Variables & Constants

A variable is a named storage location in memory. In C#, every variable must be declared with a type before use.

Variable Declaration Syntax

```
// Syntax: <datatype> <variableName> = <value>;
int studentAge = 20;
string studentName = "Priya";
double cgpa = 8.75;
bool isEnrolled = true;

// var keyword - compiler infers the type automatically
var city = "Mumbai";       // compiler sees it is a string
var marks = 95;            // compiler sees it is an int

// Constants - value cannot change after declaration
const double PI = 3.14159;
const int MAX_STUDENTS = 60;
```

2.3 Operators in C#

Operators are symbols that perform operations on values (operands).

Category	Operators	Example
Arithmetic	+ - * / % ++ --	int sum = 10 + 5; // 15
Relational	== != > < >= <=	bool result = (10 > 5); // true
Logical	&& || !	if (a > 0 && b > 0) {...}
Assignment	= += -= *= /=	x += 5; // x = x + 5
Bitwise	& | ^ ~ << >>	int result = 5 & 3; // 1
Ternary	condition ? val1 : val2	string msg = (age >= 18) ? "Adult" : "Minor";
Null Coalescing	?? ??=	string name = input ?? "Default";

2.4 Control Structures

Control structures determine the flow of execution in a program. C# supports all standard control structures.

2.4.1 If-Else Statement

Used for decision making — executes a block of code based on a boolean condition.

```
int marks = 75;
```

```
if (marks >= 90)
{
    Console.WriteLine("Grade: A+");
}
else if (marks >= 75)
{
    Console.WriteLine("Grade: A");
}
else if (marks >= 60)
{
    Console.WriteLine("Grade: B");
}
else
{
    Console.WriteLine("Grade: Fail");
}
```

2.4.2 Switch Statement

Used when you have multiple possible values for a single variable. More readable than a chain of if-else when checking equality.

```
string day = "Monday";
switch (day)
{
    case "Monday":
    case "Tuesday":
        Console.WriteLine("Weekday - Class scheduled");
        break;
    case "Saturday":
    case "Sunday":
        Console.WriteLine("Weekend - No class");
        break;
    default:
        Console.WriteLine("Unknown day");
        break;
}
```

2.4.3 Loops

Loops allow repeated execution of a block of code. C# provides four types of loops:

for Loop — use when number of iterations is known

```
// Print numbers 1 to 10
```

```
for (int i = 1; i <= 10; i++)
{
    Console.WriteLine("Number: " + i);
}
```

while Loop — use when condition is checked before execution

```
int count = 1;
while (count <= 5)
{
    Console.WriteLine("Count: " + count);
    count++;
}
```

do-while Loop — executes at least once, then checks condition

```
int num;
do {
    Console.Write("Enter a positive number: ");
    num = int.Parse(Console.ReadLine());
} while (num <= 0);
```

foreach Loop — iterate over collections (arrays, lists)

```
string[] fruits = {"Apple", "Mango", "Banana"};
foreach (string fruit in fruits)
{
    Console.WriteLine("Fruit: " + fruit);
}
```

```
LOOP SELECTION GUIDE
──────────────────────────────────────────────────────────
Do I know how many times to repeat?
  YES ──► Use FOR loop          for(int i=0; i<n; i++)
  NO  ──► Condition before?
            YES ──► WHILE loop        while(condition)
            NO  ──► DO-WHILE loop     do{ }while(cond)
Am I iterating a collection (array/list)?
  YES ──► Use FOREACH loop      foreach(var x in list)
──────────────────────────────────────────────────────────
```

2.5 Methods in C#

A method is a reusable block of code that performs a specific task. Methods help in code organization, reusability, and readability.

Method Syntax

```
// Syntax:
// <access_modifier> <return_type> <MethodName>(<parameters>)
// {
//     // method body
```

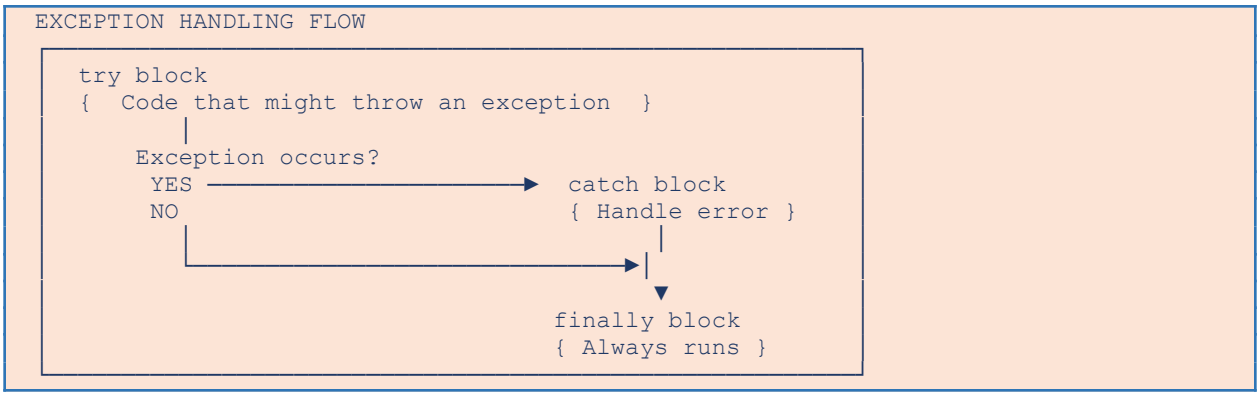
```
//     return <value>;  // if return type is not void
// }

// Example 1: Method with no parameters, no return
public void Greet()
{
    Console.WriteLine("Hello, Welcome to C#!");
}

// Example 2: Method with parameters and return value
public int AddNumbers(int a, int b)
{
    return a + b;
}

// Example 3: Static method (callable without creating object)
public static double CalculateArea(double radius)
{
    return 3.14159 * radius * radius;
}
```

Types of Parameters

Parameter Type	Syntax	Description
Value Parameter	void Method(int x)	Copy of value is passed. Changes don't affect original.
ref Parameter	void Method(ref int x)	Passes reference. Original value is modified.
out Parameter	void Method(out int x)	Must be assigned inside method. Used to return multiple values.
params Parameter	void Method(params int[] nums)	Variable number of arguments.
Optional Parameter	void Method(string s="Hi")	Has a default value if not provided by caller.

2.6 Error Handling in C#

Errors in programming fall into three categories: Syntax Errors (compile-time), Runtime Errors (exceptions), and Logic Errors (wrong output). C# provides a structured mechanism to handle runtime errors using try-catch-finally blocks.




```
try
{
    int[] arr = {1, 2, 3};
    Console.WriteLine(arr[10]);  // Index out of range!
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine("Array index error: " + ex.Message);
}
catch (Exception ex)    // Generic catch for any other exception
{
    Console.WriteLine("Unexpected error: " + ex.Message);
}
finally
{
    Console.WriteLine("This always executes - use for cleanup.");
}
```

Common Exception Types in C#

- NullReferenceException — accessing a member of a null object
- IndexOutOfRangeException — array index is out of bounds
- DivideByZeroException — dividing an integer by zero
- FormatException — converting a string to a number fails
- FileNotFoundException — specified file does not exist
- InvalidOperationException — method called in an invalid state

📌 **Note:** Custom exceptions can be created by inheriting from the Exception class. This allows you to create domain-specific error types like InvalidStudentAgeException.

Chapter 3: Object-Oriented Programming in C#

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around objects — instances of classes that bundle data (fields) and behavior (methods) together. C# was designed from the ground up as an OOP language.

```
THE FOUR PILLARS OF OOP

  1. ENCAPSULATION  ── Bundling data + methods together
  2. INHERITANCE    ── Child class inherits from parent
  3. POLYMORPHISM   ── Same action, different behavior
  4. ABSTRACTION    ── Hide complexity, show essentials
```

3.1 Classes & Objects

A Class is a blueprint (template) that defines the structure and behavior of objects. An Object is an instance (actual realization) of a class created in memory.

Class Definition & Object Creation

```
// CLASS DEFINITION
public class Student
{
    // FIELDS (data/state)
    private string name;
    private int age;
    private double cgpa;

    // CONSTRUCTOR (special method called when object is created)
    public Student(string name, int age, double cgpa)
    {
        this.name = name;   // 'this' refers to current object
        this.age = age;
        this.cgpa = cgpa;
    }

    // PROPERTIES (controlled access to fields)
    public string Name
    {
        get { return name; }
        set { name = value; }
    }

    // Auto-property shorthand
```

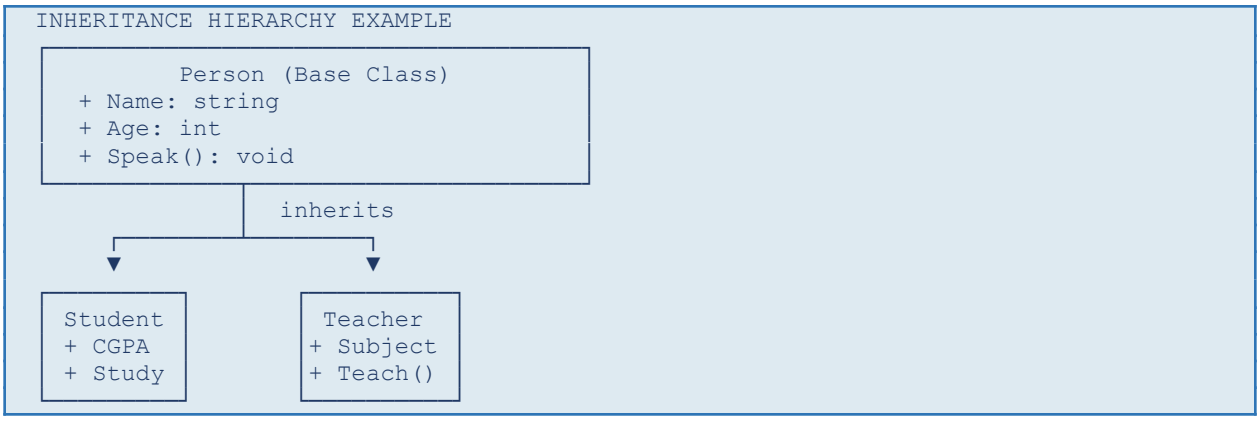
```
    public int Age { get; set; }

    // METHOD (behavior)
    public void DisplayInfo()
    {
        Console.WriteLine($"Name: {name}, Age: {age}, CGPA: {cgpa}");
    }
}

// CREATING OBJECTS
Student s1 = new Student("Rahul", 20, 8.5);
Student s2 = new Student("Priya", 21, 9.1);

s1.DisplayInfo();   // Output: Name: Rahul, Age: 20, CGPA: 8.5
s2.DisplayInfo();   // Output: Name: Priya, Age: 21, CGPA: 9.1
```

Access Modifiers

Modifier	Accessibility
public	Accessible from anywhere — any class, any assembly
private	Accessible only within the same class (default for fields)
protected	Accessible within the class and derived (child) classes
internal	Accessible only within the same assembly (.dll/.exe)
protected internal	Accessible within same assembly OR derived classes

3.2 Inheritance

Inheritance allows a class (child/derived) to acquire the properties and methods of another class (parent/base). This promotes code reuse and establishes an 'IS-A' relationship.




```
// BASE CLASS
public class Person
```

```
{
    public string Name { get; set; }
    public int Age { get; set; }

    public Person(string name, int age)
    {
        Name = name; Age = age;
    }

    public virtual void Speak()    // virtual = can be overridden
    {
        Console.WriteLine($"{Name} says: Hello!");
    }
}

// DERIVED CLASS
public class Student : Person   // ':' means 'inherits from'
{
    public double CGPA { get; set; }

    public Student(string name, int age, double cgpa)
        : base(name, age)        // calls Person's constructor
    {
        CGPA = cgpa;
    }

    public override void Speak() // override replaces parent's version
    {
        Console.WriteLine($"Student {Name} says: I am studying!");
    }
}
```

> 📌 **Note:** C# supports SINGLE inheritance (a class can have only ONE direct parent class). Multiple inheritance is achieved through Interfaces.

3.3 Polymorphism

Polymorphism means 'many forms'. It allows a single method or object to behave differently in different contexts. C# supports two types:

Compile-Time Polymorphism (Method Overloading)

Same method name, different parameters. The correct method is determined at compile time.

```
public class Calculator
{
    // Overloaded methods - same name, different parameters
    public int Add(int a, int b)              { return a + b; }
    public double Add(double a, double b)     { return a + b; }
    public int Add(int a, int b, int c)       { return a + b + c; }
    public string Add(string a, string b)     { return a + " " + b; }
}

// Usage
var calc = new Calculator();
Console.WriteLine(calc.Add(5, 3));             // calls int version
Console.WriteLine(calc.Add(3.14, 2.71));       // calls double version
```

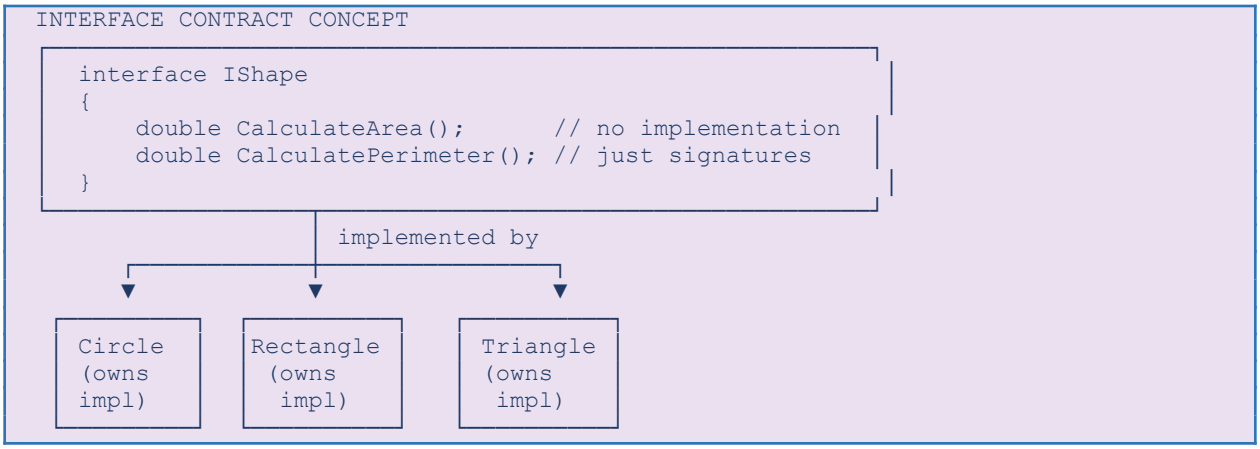
Runtime Polymorphism (Method Overriding)

A derived class provides its own implementation of a method declared in the base class using virtual and override keywords. Determined at runtime.

```
// The Speak() method from the Inheritance example above
// demonstrates runtime polymorphism:

Person p = new Student("Amit", 20, 8.9);  // Person reference, Student object
p.Speak();  // Calls Student's Speak(), not Person's!
            // Output: Student Amit says: I am studying!
            // This is RUNTIME polymorphism - decided at runtime
```

3.4 Interfaces

An Interface is a contract — it defines a set of methods/properties that a class MUST implement. Interfaces enable multiple inheritance in C# and are fundamental to good software design (Dependency Inversion Principle).




```
// INTERFACE DEFINITION
```

```
public interface IShape
{
    double CalculateArea();
    double CalculatePerimeter();
    string ShapeName { get; }  // properties allowed too
}

// CLASS IMPLEMENTING INTERFACE
public class Circle : IShape
{
    private double radius;
    public Circle(double r) { radius = r; }

    public string ShapeName => "Circle";
    public double CalculateArea() => 3.14159 * radius * radius;
    public double CalculatePerimeter() => 2 * 3.14159 * radius;
}

// A class can implement MULTIPLE interfaces
public class SmartDevice : IShape, IDisplayable, IPrintable
{
    // must implement all methods from all three interfaces
}
```

Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	Only signatures (C#8+ can have default)	Can have abstract + concrete methods
Fields	Not allowed	Allowed
Multiple	A class can implement many	Only one base class allowed
Constructor	Not allowed	Allowed
Use When	Define a capability/contract	Share common logic among related classes

3.5 Namespaces

A Namespace is a container that organizes code into logical groups and prevents naming conflicts. Think of it like folders in a file system — two files with the same name can exist in different folders.

```
// Defining a namespace
namespace CollegeManagement.Academics
{
```

```
    public class Student { /* ... */ }
    public class Course  { /* ... */ }
}

namespace CollegeManagement.Finance
{
    public class Student { /* different Student class! */ }
    public class FeeRecord { /* ... */ }
}

// Using namespaces
using CollegeManagement.Academics;
using CollegeManagement.Finance;

// Access with full path if names conflict:
CollegeManagement.Academics.Student s1 = new ...;
CollegeManagement.Finance.Student s2 = new ...;
```

Common Built-in .NET Namespaces

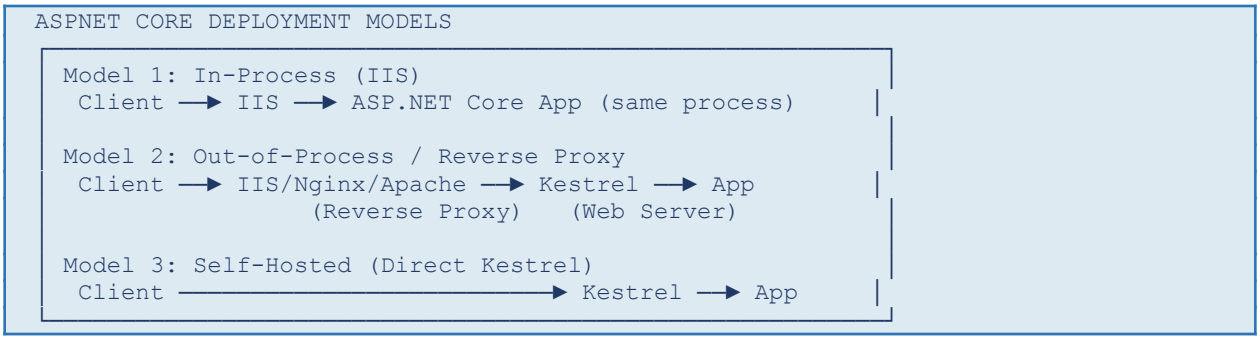
Namespace	Contains
System	Console, Math, String, Object, Exception — core types
System.Collections.Generic	List<T>, Dictionary<K,V>, Queue<T>, Stack<T>
System.IO	File, Directory, StreamReader, StreamWriter
System.Linq	LINQ query methods for collections
System.Net.Http	HttpClient for making web requests
System.Threading.Tasks	Task, async/await for asynchronous programming

Chapter 4: ASP.NET Core Fundamentals

ASP.NET Core is the framework used to build web applications and APIs using C#. It is cross-platform, high-performance, and open-source. It is the successor to the original ASP.NET and supports web apps, REST APIs, real-time apps (SignalR), and microservices.

4.1 Web Server Basics & Kestrel

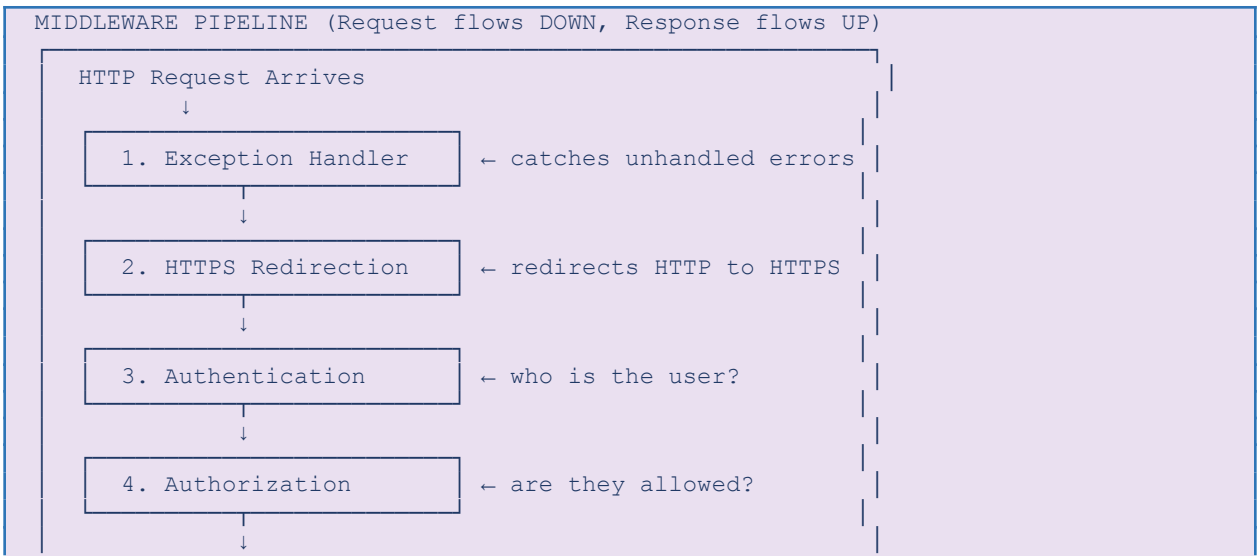
When you build an ASP.NET Core application, it runs on a built-in web server called Kestrel. Kestrel is a cross-platform, asynchronous HTTP server that is included with ASP.NET Core.

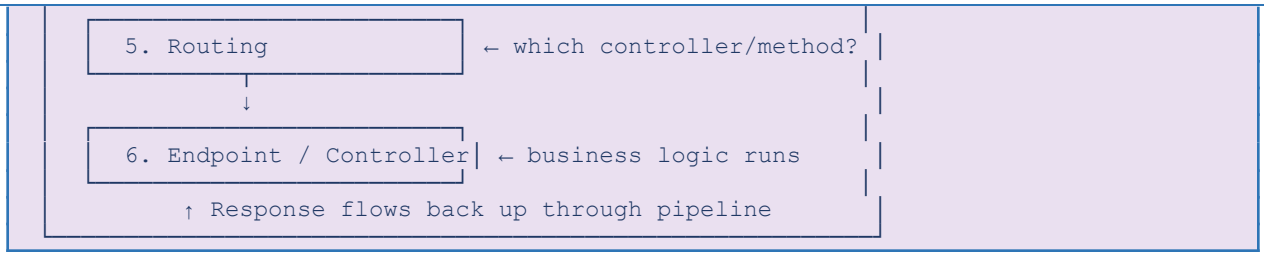


4.2 Middleware

Middleware are software components that are assembled into an application pipeline to handle requests and responses. Each middleware component can: process the request, pass it to the next middleware, or short-circuit the pipeline.

> 📌 **Note:** Think of middleware as a series of checkpoints or filters through which every HTTP request passes before reaching your controller/endpoint.



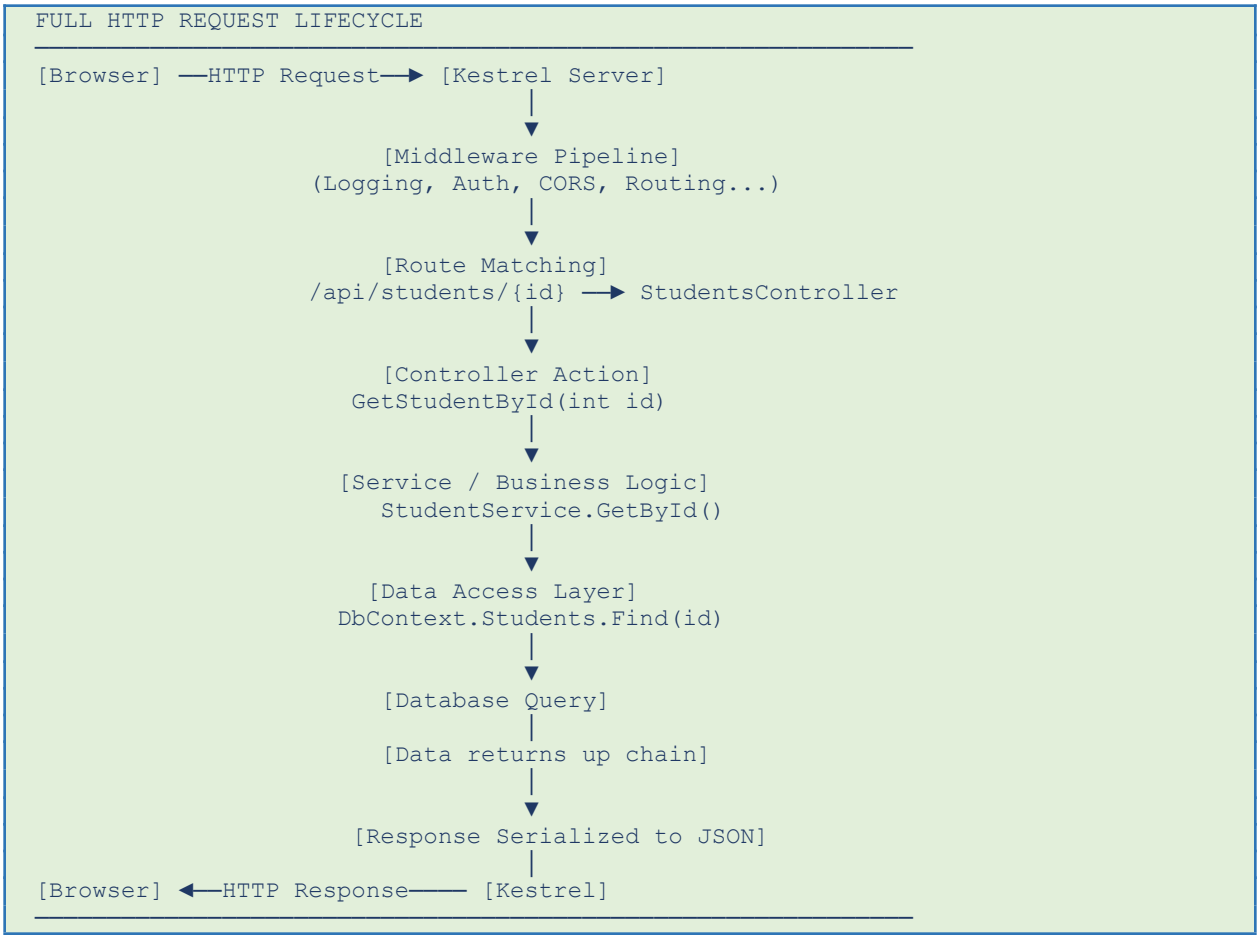



```
// Program.cs - Middleware Registration
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseExceptionHandler("/Error");  // Add exception middleware
app.UseHttpsRedirection();           // Redirect HTTP to HTTPS
app.UseStaticFiles();               // Serve static files (HTML/CSS/JS)
app.UseRouting();                   // Enable routing
app.UseAuthentication();            // Check user identity
app.UseAuthorization();             // Check user permissions
app.MapControllers();               // Map controller routes

app.Run();    // Start the web server
```

Creating Custom Middleware

```
// Method 1: Inline middleware using app.Use()
app.Use(async (context, next) =>
{
    Console.WriteLine("Request: " + context.Request.Path);
    await next();    // Pass to next middleware
    Console.WriteLine("Response: " + context.Response.StatusCode);
});

// Method 2: Class-based middleware
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;
    public LoggingMiddleware(RequestDelegate next) { _next = next; }

    public async Task Invoke(HttpContext context)
    {
        Console.WriteLine($"[{DateTime.Now}] {context.Request.Method}
{context.Request.Path}");
        await _next(context);
    }
}
// Register: app.UseMiddleware<LoggingMiddleware>();
```

4.3 Request Lifecycle in ASP.NET Core



4.4 Routing in ASP.NET Core

Routing is the process of matching an incoming HTTP request URL to a specific controller action or endpoint. ASP.NET Core supports two types of routing:

Convention-Based Routing

Routes are defined once in a central location using patterns.

```
// Convention-based routing setup
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
// Examples of URL matching:
// /Home/Index        → HomeController.Index()
// /Student/List      → StudentController.List()
// /Student/Details/5 → StudentController.Details(5)
```

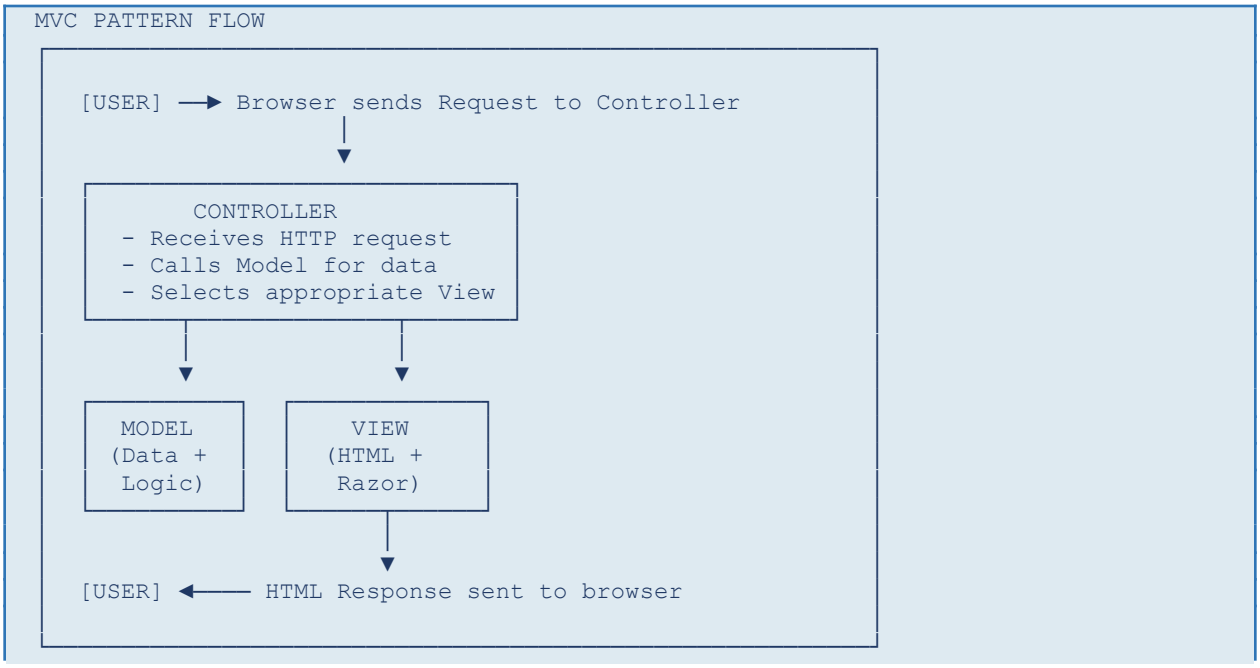
Attribute Routing

Routes are defined directly on the controller and action using attributes. Preferred for REST APIs.

```
[ApiController]
[Route("api/[controller]")]   // base route: api/students
public class StudentsController : ControllerBase
{
    [HttpGet]                 // GET  api/students
    public IActionResult GetAll() { ... }

    [HttpGet("{id}")]         // GET  api/students/5
    public IActionResult GetById(int id) { ... }

    [HttpPost]                // POST api/students
    public IActionResult Create([FromBody] Student s) { ... }

    [HttpPut("{id}")]         // PUT  api/students/5
    public IActionResult Update(int id, [FromBody] Student s) { ... }

    [HttpDelete("{id}")]      // DELETE api/students/5
    public IActionResult Delete(int id) { ... }
}
```

4.5 Controllers & Views (MVC Pattern)

MVC stands for Model-View-Controller. It is a design pattern that separates an application into three interconnected components:



```
M = Model (data/business logic)
V = View  (what user sees - HTML/Razor)
C = Controller (handles requests & coordinates M and V)
```

Creating a Controller

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    // GET /Home/Index
    public IActionResult Index()
    {
        ViewBag.Title = "Welcome";
        ViewBag.Message = "Hello from ASP.NET Core MVC!";
        return View();  // Returns Views/Home/Index.cshtml
    }

    // GET /Home/About
    public IActionResult About()
    {
        var model = new AboutViewModel { CompanyName = "ABC Corp" };
        return View(model);  // Passes model to view
    }
}
```

4.6 Razor Pages

Razor Pages is an alternative to MVC that is simpler for page-focused scenarios. Each page has a .cshtml file (the view) and a .cshtml.cs file (the PageModel — combines model + controller logic).

Razor Syntax (used in both MVC Views and Razor Pages)

```
@* This is a Razor comment *@

@{
    var title = "Student List";
    var count = Model.Students.Count;
}

<h1>@title</h1>          @* @ renders C# variable *@
<p>Total: @count students</p>

@* Conditional rendering *@
@if (Model.IsLoggedIn)
```

```
{
    <p>Welcome, @Model.UserName!</p>
}
else
{
    <a href="/login">Please log in</a>
}

@* Loop rendering *@
<ul>
@foreach (var student in Model.Students)
{
    <li>@student.Name - CGPA: @student.CGPA</li>
}
</ul>
```

4.7 REST API Endpoints with ASP.NET Core

REST (Representational State Transfer) is an architectural style for building web services. REST APIs use HTTP methods to perform CRUD operations:

HTTP Method	CRUD Operation	Example URL	What It Does
GET	Read	GET /api/students	Retrieve all students
GET	Read	GET /api/students/5	Retrieve student with ID 5
POST	Create	POST /api/students	Create a new student
PUT	Update (Full)	PUT /api/students/5	Replace student 5 completely
PATCH	Update (Partial)	PATCH /api/students/5	Update specific fields only
DELETE	Delete	DELETE /api/students/5	Remove student 5

Complete REST API Controller Example

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly IStudentService _service;

    public StudentsController(IStudentService service)
    {
        _service = service;  // Dependency Injection
    }
```

```
    // GET api/students
    [HttpGet]
    public async Task<ActionResult<List<Student>>> GetAll()
    {
        var students = await _service.GetAllAsync();
        return Ok(students);      // 200 OK with JSON data
    }

    // GET api/students/5
    [HttpGet("{id}")]
    public async Task<ActionResult<Student>> GetById(int id)
    {
        var student = await _service.GetByIdAsync(id);
        if (student == null)
            return NotFound();    // 404 Not Found
        return Ok(student);
    }

    // POST api/students
    [HttpPost]
    public async Task<ActionResult<Student>> Create([FromBody] Student
student)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);  // 400 Bad Request
        var created = await _service.CreateAsync(student);
        return CreatedAtAction(nameof(GetById), new { id = created.Id },
created); // 201
    }

    // DELETE api/students/5
    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var success = await _service.DeleteAsync(id);
        if (!success) return NotFound();
        return NoContent();       // 204 No Content
    }
}
```

4.8 HTTP Status Codes Reference

Code	Name	When to Use

200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE or PUT with no response body
400	Bad Request	Invalid input / validation failed
401	Unauthorized	Not authenticated (login required)
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource does not exist
500	Internal Server Error	Unhandled server-side error

4.9 Web Forms (Legacy)

Web Forms is the original ASP.NET programming model (introduced in 2002). It abstracts HTTP and HTML behind a drag-and-drop UI model similar to desktop application development (like Windows Forms).

Aspect	Web Forms	ASP.NET Core MVC/API
Programming Model	Event-driven (like WinForms)	Request-response (HTTP-centric)
Page Lifecycle	Complex (Init, Load, PostBack, Render...)	Simple (Request → Action → Response)
ViewState	Uses ViewState for state management (heavy)	Stateless by default (lightweight)
HTML Control	Limited (server controls add extra HTML)	Full control over HTML output
Testability	Difficult to unit test	Highly testable
Status	Legacy - not in .NET Core	Current recommended approach

> 📌 **Note:** Web Forms is only available in the old .NET Framework (not .NET Core / .NET 5+). For new projects, always use ASP.NET Core MVC or Razor Pages. Understanding Web Forms is important for maintaining legacy enterprise applications.

Module 1 Summary — Key Concepts at a Glance

Chapter	Core Takeaways
Ch 1: .NET Ecosystem	.NET is a platform. CLR runs code. Assemblies are compiled output. Use CLI for development. .NET 8 is the current LTS.
Ch 2: C# Basics	Strong-typed language with value & reference types. Control flow using if/switch/loops. Methods are reusable code blocks. Error handling via try-catch.
Ch 3: OOP in C#	4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction. Classes are blueprints. Interfaces are contracts. Namespaces organize code.
Ch 4: ASP.NET Core	Web server (Kestrel) + Middleware pipeline + Routing + MVC pattern + Razor + REST APIs (CRUD via HTTP methods).

End of Module 1 Notes